Installing and Running CoLoRe on NERSC

Dani L. Jarque

Updated October 2026

1 Overview

These notes describe a practical setup for compiling and running CoLoRe on NERSC systems. The main points are to install CoLoRe's non-system dependencies in a dedicated Conda environment, load NERSC's Conda and FFTW modules both when compiling and when running, compile against the intended libraries, and make the Conda environment's shared libraries visible at runtime.

NERSC modules, compilers, and library versions can change. Treat paths and module names below as a starting point and check current NERSC documentation if a module or path is unavailable. Customize account names, environment names, repository locations, simulation directories, and Slurm resources for your own setup.

2 Create a Conda environment

Start a login shell on NERSC and load the Conda module. Create a dedicated environment and install the libraries used by CoLoRe:

```
module load conda
conda create -n colore_env
conda activate colore_env

conda install -c conda-forge libconfig libsharp cfitsio chealpix
```

If your existing environment already contains these libraries, you do not need to create a second one. Check the active environment and installed packages with:

```
echo "$CONDA_PREFIX"
conda list
```

Keep the environment name consistent in the commands below. If you choose a different name, replace `colore_env` accordingly.

3 Load FFTW and set library paths

CoLoRe also uses FFTW. On the NERSC setup used for these instructions, load the Cray FFTW module:

```
module load cray-fftw
```

The module provides FFTW headers and libraries for compilation. The exact installation path depends on the NERSC system and loaded module version. If the Makefile needs an explicit FFTW path, use the path reported by the loaded module or documented for your system. For example, this path was used on one NERSC CPU setup, but should be checked before reuse:

```
export FFTW_DIR=/opt/cray/pe/fftw/3.3.10.11/x86_milan
```

For runtime linking, add the active Conda environment's library directory to `LD_LIBRARY_PATH`. Do this after activating the environment:

```
export LD_LIBRARY_PATH="$CONDA_PREFIX/lib:${LD_LIBRARY_PATH:-}"
```

This uses the active environment rather than a hard-coded path. Run the same command in your interactive shell when testing and in every batch script that runs CoLoRe.

4 Get the CoLoRe source code

Clone the repository using its Git URL (not the URL of the web interface's `tree` page):

```
cd "$HOME"
git clone https://github.com/damonge/CoLoRe.git
cd CoLoRe
```

If you need a particular branch or revision, check it out explicitly before compiling. Record the branch or commit used for production runs so the executable can be reproduced later.

5 Configure the Makefile

Open the repository's `Makefile` and inspect the user-configurable section near the top. The exact variable names and supported options can vary between CoLoRe revisions, so use the Makefile shipped with the source version you cloned rather than replacing the entire file with an example from an older installation guide.

For a typical NERSC CPU build, check the following:

- Use compiler wrappers and options supported by the current NERSC programming environment. The compiler setting must match the compiler modules available on the system.
- Enable MPI and OpenMP if your run is intended to use both.
- Enable FITS support if your input/output workflow requires it.
- Set include and library paths for FFTW and the Conda-installed libraries. Prefer paths based on `${CONDA_PREFIX}` over an absolute path belonging to a specific user.
- Choose the desired bias model using the preprocessor flags documented in the source. Do not enable mutually exclusive bias-model flags together.
- Enable HDF5 only if the build has the required HDF5 development libraries and you intend to use that capability.

For example, if the Makefile uses variables for the Conda-installed libraries, their paths will generally have this form:

```
CONDA_DIR = /global/common/software/desc-wl/djarque/conda/envs/colore_env

#GSL
GSL_INC = -I$(CONDA_DIR)/include
GSL_LIB = -L$(CONDA_DIR)/lib

#FFTW
FFTW_INC = -I$(CONDA_DIR)/include
FFTW_LIB = -L$(CONDA_DIR)/lib

#cfitsio
FITS_INC = -I$(CONDA_DIR)/include
FITS_LIB = -L$(CONDA_DIR)/lib

#libconfig
CONF_INC = -I$(CONDA_DIR)/include
CONF_LIB = -L$(CONDA_DIR)/lib

...
```

These are examples of path settings, not a replacement for the full Makefile: check the variable names, link flags, and library names used by your CoLoRe revision. Some installations of `libsharp` or `chealpix` may place files in subdirectories or use different library names. If compilation cannot find a header or the linker cannot find a library, inspect the files under `${CONDA_PREFIX}` and adjust the relevant Makefile entry.

Do not copy old system-wide GSL paths blindly. Use the module or library installation appropriate for your NERSC system and the source version being compiled.

6 Compile CoLoRe

Load the required modules and activate the environment in the same shell used for compilation:

```
module load conda
module load cray-fftw
conda activate colore_env

export LD_LIBRARY_PATH="$CONDA_PREFIX/lib:${LD_LIBRARY_PATH:-}"
cd "$HOME/CoLoRe"
make
```

If you change build flags or dependencies after a previous build, clean the old objects and rebuild. Check the Makefile for the supported clean target; for versions that provide it, use:

```
make clean
make
```

Compilation is successful only if `make` exits without an error and produces the expected executable. Check the executable name and location in your repository; do not assume every branch produces a binary at the same path.

Before submitting a large job, test that the executable starts and that its shared libraries resolve. On Linux, `ldd /path/to/CoLoRe` is useful for diagnosing missing shared libraries. If it reports a library as "not found", check that the relevant module is loaded, the intended Conda environment is active, and `LD_LIBRARY_PATH` includes the appropriate library directory. Do not create arbitrary symbolic links between incompatible library versions to silence linker errors.

7 Run CoLoRe with Slurm

The batch script must load the modules, activate the same environment, and set the runtime library path before launching CoLoRe. This template is based on a working CPU job. Replace the executable path, configuration path, output path, account, and resource requests with your own values.

```
#!/bin/bash -l

#SBATCH --account YOUR_ACCOUNT
#SBATCH --job-name colore_run
#SBATCH --qos debug
#SBATCH --constraint cpu
#SBATCH --nodes 8
#SBATCH --ntasks-per-node 8
#SBATCH --cpus-per-task 32
#SBATCH --time 00:08:00
#SBATCH --output %x.out
#SBATCH --error %x.err

module load conda
module load cray-fftw
conda activate colore_env
```

```
# Customize these paths for your project.
COLORE="$HOME/CoLoRe/CoLoRe"
config="$PSCRATCH/path/to/param.cfg"
results="$PSCRATCH/path/to/results"

# Make the active Conda environment's shared libraries available at runtime.
export LD_LIBRARY_PATH="$CONDA_PREFIX/lib:${LD_LIBRARY_PATH:-}"

# OpenMP configuration: keep consistent with the Slurm allocation.
export OMP_NUM_THREADS=32
export OMP_PLACES=threads
export OMP_PROC_BIND=spread

date
hostname

mkdir -p "$results"
srun "$COLORE" "$config"

date
```

The example allocates 8 nodes, 8 MPI tasks per node, and 32 CPUs per task. These values are not a universal recommendation: choose resources to match the parallel layout and memory needs of your simulation. The `debug` queue and short walltime are intended for short tests; use the appropriate NERSC queue and walltime for production runs, following current allocation policies.

The script creates the directory stored in `results`, but CoLoRe's actual output location is controlled by the configuration file. Ensure the output prefix and paths in `param.cfg` match the directory structure you intend to use.

8 Troubleshooting checklist

If CoLoRe fails to compile or start, check these items in order:

1. **Modules:** confirm that `conda` and `cray-fftw` are loaded in the shell or batch job.
2. **Environment:** confirm that `conda activate colore_env` succeeds and that `CONDA_PREFIX` points to the intended environment.
3. **Runtime paths:** run `export LD_LIBRARY_PATH="${CONDA_PREFIX}/lib:${LD_LIBRARY_PATH:-}"` after activating Conda.
4. **Executable:** verify that the batch script points to the executable you just compiled.
5. **Dependencies:** use `ldd /path/to/CoLoRe` to identify missing shared libraries, and resolve each missing dependency using the appropriate module or compatible library installation.
6. **Configuration:** check that the configuration file exists and that all input files and output paths are accessible from compute nodes.
7. **Slurm layout:** ensure the MPI task count, OpenMP thread count, and allocated CPUs are consistent with the intended run.

Important reminders

- Load modules and activate the Conda environment in every batch script; an interactive login shell's environment should not be assumed to carry over.
- If you change a compile-time option, such as the bias model or precision, recompile CoLoRe and keep the executable associated with that configuration.
- Record the repository revision, Makefile changes, environment packages, and run configuration for reproducibility.
- Avoid hard-coded paths containing another user's username or Conda environment. Use `$HOME`, `$PSCRATCH`, and `$CONDA_PREFIX` where appropriate.